PRAKTIKUM 5
PROCEDURE & FUNCTION

5.1 Tujuan

1. Mahasiswa mampu memanfaatkan *function* yang sudah disediakan oleh Code Block.
2. Mahasiswa mampu membuat *function* dengan memanfaatkan variabel lokal maupun global.
3. Mahasiswa mampu membuat *function* dengan menggunakan parameter *input* dan parameter *output*.
4. Mahasiswa mampu membuat *function* yang mengembalikan dan tidak mengembalikan nilai.

5.2 Landasan Teori

5.2.1 Pengertian

Procedure dan *Function* dalam bahasa pemrograman biasa juga disebut dengan Sub *Routine*. Secara garis besar, *Procedure* dan *Function* memiliki pengertian yang sama, yaitu merupakan sebuah blok *statement* yang dapat dipanggil lebih dari 1 kali dari lokasi yang berbeda dalam sebuah program (UNY, n.d.). Dalam beberapa bahasa pemrograman pun, mereka dianggap sama. Namun, ada pula yang menganggap mereka berbeda. Seperti dalam bahasa PASCAL, mereka dianggap berbeda, dikenal dengan *Procedure* dan *Function*. Dalam bahasa BASIC, mereka dikenal dengan *Sub* dan *Function*. Sedangkan dalam bahasa C++, Java, PHP, dan keturunan bahasa C lainnya, mereka hanya mengenal *Function* saja. Namun jika dibandingkan betul-betul, sebenarnya terdapat perbedaan diantara keduanya. Yakni *Function* merupakan sebuah blok program yang jika dipanggil dan dijalankan akan mengembalikan suatu nilai, sedangkan *Procedure* tidak mengembalikan sebuah nilai.

Bagaimana dengan C++, Java, PHP, dan berbagai bahasa pemrograman turunan C lainnya yang tidak mengenal *Procedure*? Kita dapat memanfaatkan sintaks `void` untuk tipe data kembalian dari *Function*, yang berarti *Function*

tersebut tidak memiliki nilai kembalian. Karena kita akan membahas C++, maka selanjutnya kita akan menyebut *Procedure* dan *Function* sebagai *Function* saja.

Berikut merupakan kegunaan *Function*:

1. Lebih ringkas, mengurangi penulisan program berkali-kali, karena *Function* bisa dipanggil lebih dari satu kali.
2. Dapat melakukan pendekatan *top-down* dan *divide-and-conquer*, yaitu sebuah program yang besar namun dipisah menjadi bagian-bagian kecil program.
3. Program menjadi lebih mudah dipahami karena menjadi lebih tertata dan terstruktur.
4. Mudah dalam melakukan *troubleshooting* karena alur program menjadi lebih jelas sehingga akan lebih mudah menemukan letak kesalahan apabila terdapat kesalahan dalam jalannya program. Yang diperbaikipun lebih jelas dan hanya pada modul tertentu saja tanpa merubah program keseluruhan
5. Mempermudah proses pembuatan dokumentasi
6. *Function* menjadi *reuseability*, dalam satu program, *function* dapat dipanggil (digunakan) berkali-kali.

Dalam bahasa C, *function* dapat dibagi menjadi dua, yaitu:

a) *Standard Library Function*: yaitu *function* yang ada di dalam *header* atau *library* yang telah disediakan oleh C/C++. Sehingga apabila ingin memanfaatkan salah satu *Function* tersebut, maka harus mendeklarasikan (preprosesor direktif) *header* atau *library* nya dahulu.

   Contoh: apabila ingin memanggil `system("cls);` harus mendeklarasikan terlebih dahulu `library #include <stdlib.h>` di paling awal program.

b) *Programmer-Defined Function*: yaitu *Funtion* yang diciptakan sendiri oleh pemrogram. Memiliki nama yang unik dan terletak terpisah dari program utama. Bisa dalam satu program atau dimasukkan ke dalam *library* buatan pemrogram sendiri yang kemudian di deklarasikan terlebih dahulu apabila ingin menggunakan *Function* tersebut.

5.2.2 Format Function

Sintaks 5.1 merupakan format penulisan *Function*:

```
tipe_data nama_function (tipe_data argument1, tipe_data
argument2, ...) {
     statement
}
```

Sintaks 5. 1 *Format Penulisan Funtion*

Dimana:

a) `tipe_data`, adalah tipe data dari nilai yang akan dikembalikan atau nilai hasil dihasilkan oleh proses dalam *Function*.

b) `nama_function`, adalah nama *Function* yang bersifat unik.

c) `tipe_data argument1, tipe_data argument2` (sesuai kebutuhan) merupakan deklarasi parameter yang akan digunakan dalam *Function*. Setiap `argument` dideklarasikan oleh tipe data dan variabel (contoh: int x).

d) `statement`, merupakan bagian utama dalam *Function*. Boleh berupa instruksi tunggal atau satu blok instruksi yang dituliskan diantara kurung kurawal {}.

Contoh penulisan fungsi:

```
float luas_lingkaran(float jari)
{
      return (3.14 * jari * jari);
}
```

Sintaks 5. 2 *Contoh Penulisan Function*

5.2.3 Struktur *Function*

Ada 2 cara yang bisa dilakukan pemrogram dalam penulisan *Function*. Yaitu *Programmer-Defined Function* ditulis sebelum atau sesudah *Function* `main()`. Sehingga ada 2 cara pula dalam pendeklarasian *Function*:

a) Tanpa deklarasi

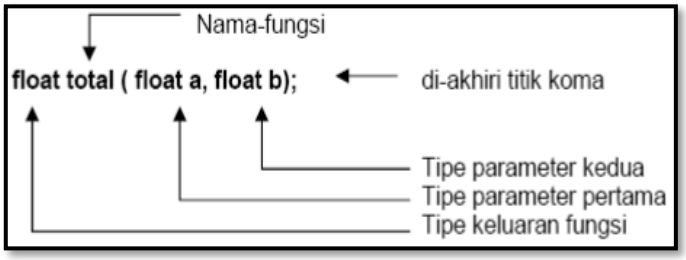
Function tidak perlu dideklarasikan apabila *Function* berada di atas *Function* utama, main().

b) *Function Prototype*

Prototype Function digunakan untuk mendeklarasikan atau mengenalkan sebuah *Function* kepada *compiler* apabila *Programmer-Defined Function* terletak di bawah *Function* `main()`.

Salah satu keuntungan apabila pemrogram menggunakan *Prototype Function* adalah *compiler* akan membandingkan tipe parameter dalam *prototype* dan parameter pada *Function* saat pemanggilan *Function*. Program akan menunjukkan adanya kesalahan apabila tidak sama atau jika jumlah parameter dalam *prototype* dan pada saat pemanggilan berbeda.

Contoh penulisan *Prototype Function*:



Sumber : (Afandi, 2011)

Gambar 5. 1 *Contoh Penulisan Prototype Function*

Contoh lain penulisan *Prototype Function*:

a) `long kuadrat (long l) ;`
b) `void garis ();`
c) `double maks (double x, double y) ;`

Function kuadrat ( ) pada contoh 1 mempunyai parameter bertipe data long dan memiliki nilai balik bertipe long.

Pada contoh 2, *Function* garis ( ) tidak memiliki parameter dan tidak memiliki nilai balik (`void`).

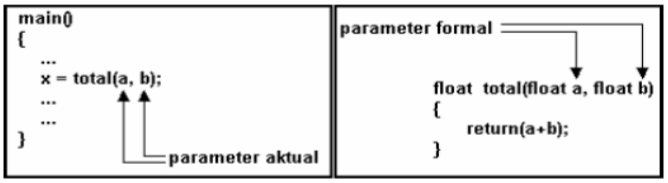
Pada contoh 3, *Function* maks ( ) mempunyai dua buah parameter, dengan masing-masing parameter bertipe data *double* dan memiliki nilai balik bertipe *double*.

5.2.4 Function Parameter

Terdapat dua macam parameter dalam *Function*, yaitu:

1. Parameter Formal: variabel yang ada pada parameter *Function*.
2. Parameter Aktual: variabel yang digunakan dalam pemanggilan *Function*.

Gambar 5.2 merupakan bentuk penulisan Parameter Formal dan Parameter Aktual:



Sumber : (Afandi, 2011)

Gambar 5. 2 *Bentuk Penulisan Parameter Formal dan Parameter Aktual*

Contoh program:

```
#include <stdio.h>

float L_persegipanjang(float p, float l); //prototype
untuk Function L_persegipanjang
float L_lingkaran(float r); //prototype untuk Function
L_lingkaran
//parameter formal pada fungsi diatas adalah variabel: p,
l dan r

int main (){
     float pj = 20.25;
     float lb = 4.75;
     float ls;
     ls = L_persegipanjang(pj,lb); //parameter aktual
     berupa variabel pj dan lb
     printf("HASIL HITUNG LUAS PERSEGI PANJANG = f\n",ls);
     ls = L_lingkaran(10); //parameter aktual berupa
     konstanta 10
     printf("HASIL HITUNG LUAS LINGKARAN = %f \n",ls);
     return 0;
}

 //definisi Function L_persegipanjang dan L_lingkaran
float L_persegipanjang(float pj, float lb){
     return (pj * lb);
}
float L_lingkaran(float r)    {
     return (3.14 * r * r );
}
```

Sintaks 5. 3 *Contoh Program Penggunaan Parameter Formal dan Aktual*

Tampilan program ketika dijalankan:

```
HASIL HITUNG LUAS PERSEGI PANJANG = f
HASIL HITUNG LUAS LINGKARAN = 314.000000

Process returned 0 (0x0)   execution time : 0.190 s
Press any key to continue.
```

Gambar 5. 3 *Tampilan program ketika Sintaks 5.3 Dijalankan*

5.2.5 Jenis Function

Terdapat 2 jenis *Function* berdasarkan bentuknya:

a. *Function* yang Mengembalikan Nilai

Yang dimaksud dengan *Function* yang mengembalikan nilai adalah suatu *Function* yang menghasilkan nilai ketika *Function* tersebut dieksekusi. Oleh sebab itu, sebuah *Function* yang mengembalikan nilai harus mempunyai tipe data yang sesuai dengan nilai yang dihasilkannya.

Perhatikan contoh berikut:

```
#include<iostream>

using namespace std;

double gaya(double m, double a)
{
     double hasil;
     hasil=m*a;

     return (hasil);   //memberikan nilai balik bertipe data
     double
}

int main() {
     double m,a,f;

     cout<<"Massa      : ";
     cin>>m;

     cout<<"percepatan: ";
     cin>>a;

     f=gaya(m,a);
     cout<<"F: "<<f;

     return 0;
}
```

Sintaks 5. 4 *Contoh Function yang Mengembalikan Nilai*

Tampilan program ketika dijalankan:

```
Massa      : 20
percepatan: 10
F: 200
Process returned 0 (0x0)   execution time : 7.701 s
Press any key to continue.
```

Gambar 5. 4 *Tampilan program ketika Sintaks 5.4 Dijalankan*

Function `double gaya()` di atas merupakan *Function* yang mengembalikan nilai bertipe data *double*. Bisa dibuktikan dengan adanya tipe data di depan nama *Function*. Jika dilihat dari contoh di atas, ketika *Function* `gaya()` dipanggil melalui `main()`, hasil nilai balik dari *Function* `gaya()` disimpan pada variabel `f`.

Sehingga dapat disimpulkan bahwa ciri dari *Function* yang mengembalikan nilai adalah:

i. *Function* tersebut memiliki tipe data yang ditulis di depan nama *Function*.

ii. Di akhir blok *Function*, terdapat sintaks `return` yang berfungsi untuk *passing* nilai.

b. *Function* yang Tidak Mengembalikan Nilai

Function yang tidak mengembalikan nilai sebenarnya secara bentuk sama saja dengan *Function* yang mengembalikan nilai. Perbedaannya hanya terletak pada penggunaan sintaks `void` pada *Function* yang tidak mengembalikan nilai.

Berikut merupakan contoh *Function* yang tidak mengembalikan nilai:

```
#include<iostream>

using namespace std;

void gaya(double m, double a)
{
    double hasil;
    hasil=m*a;
    cout<<hasil;
}

int main()
{
    double m,a;
    cout<<"Massa     : ";

    cin>>m;
    cout<<"percepatan: ";

    cin>>a;
    cout<<"F: ";

    gaya(m,a);
    return 0;
}
```

Sintaks 5. 5 *Contoh Function yang Tidak Mengembalikan Nilai*

Tampilan program ketika dijalankan:

```
Massa     : 10
percepatan: 20
F: 200
Process returned 0 (0x0)   execution time : 23.374 s
Press any key to continue.
```

Gambar 5. 5 *Tampilan program ketika Sintaks 5.5 Dijalankan*

Berdasarkan contoh pada sintaks 5.5, *Function* `main()` hanya memanggil *Function* `gaya()` dan kemudian proses dikendalikan oleh *Function* `gaya()`. Setelah *Function* `gaya()` memproses perhitungan, *Function* `gaya()` kemudian tidak mengembalikan hasil tersebut ke *Function* `main()` yang memanggilnya melainkan ditampilkan sendiri oleh *Function* `gaya()`.

5.2.6 Cara Pelewatan Parameter *Function*

a. Pelewatan Parameter melalui Nilai (*Pass by value*)

Pemanggilan dengan nilai adalah cara pelewatan nilai untuk parameter dengan cara melemparkan nilai melalui parameter aktual. Sehingga nilai dari parameter aktual akan tersalin secara otomatis ke parameter formal. Dengan cara tersebut, maka nilai pada parameter aktual tidak akan berubah ketika parameter formal berubah.

Contoh:

```
#include <iostream>

using namespace std;

int pangkat(int x)
{
     x = x*x;
     return(x);
}

int main(void)
{
     int n=10;

     cout << "Hasil " <<   n   <<   "x"   <<   n   <<   "="
     << pangkat(n) <<endl;

     return 0;
}
```

Sintaks 5. 6 *Contoh Pelewatan Parameter Melalui Nilai*

Tampilan program ketika dijalankan:

```
Hasil 10x10=100

Process returned 0 (0x0)   execution time : 0.275 s
Press any key to continue.
```

Gambar 5. 6 *Tampilan program ketika Sintaks 5.6 Dijalankan*

Nilai `parameter` aktual yang diberikan untuk *Function* `pangkat()` pada *Function* `main()` melalui variabel `n` (bernilai 10) disalin ke parameter `x` (parameter formal) pada *Function* `pangkat()`. Selanjutnya, didalam *Function* `pangkat()`, nilai `x` akan diolah dan berubah menjadi 100 (10x10). Setelah itu, hasil perhitungan

tersebut dikeluarkan dari *Function* `pangkat()`, dan diterima oleh *Function* `main()` untuk selanjutnya langsung ditampilkan. Nilai `n` yang ditampilkan akan tetap bernilai 10, sedangkan hasil keluaran dari sintaks `pangkat(n)` adalah `100`.

b. Pelewatan Parameter melalui Referensi (*Pass by Reference*)

Pelewatan parameter melalui referensi adalah pelewatan parameter dengan menggunakan variabel referensi. Sehingga, nilai pada *Function* akan sama dengan nilai pada *Function* pemanggil. Hal tersebut dikarenakan alamat yang digunakan sebagai alamat untuk parameter aktual sama dengan alamat yang digunakan sebagai alamat parameter formal.

Contoh:

```
#include <iostream>

using namespace std;

void tukar(int *x, int *y) {
    int z;
    z = *x;
    *x = *y;
    *y = z;

    cout<<endl;
    cout<<"Nilai di Akhir Function Tukar()"<<endl;
    cout<<"x = "<<*x<<" y = "<<*y<<endl;
}

int main() {
    int i, j;
    i = 10;
    j = 20;

    cout<<"Nilai Sebelum Function Dipanggil"<<endl;
    cout<<"i = "<<i<<" j = "<<j<<endl;

    tukar(&i,&j);

    cout<<endl;
    cout<<"Nilai Setelah Function Dipanggil"<<endl;
    cout<<"i = "<<i<<" j = "<<j<<endl;

    return 0;
}
```

Sintaks 5. 7 *Contoh Function yang Mengembalikan Referensi*

Tampilan program ketika dijalankan:

```
Nilai Sebelum Function Dipanggil
i = 10 j = 20

Nilai di Akhir Function Tukar()
x = 20 y = 10

Nilai Setelah Function Dipanggil
i = 20 j = 10

Process returned 0 (0x0)   execution time : 0.322 s
Press any key to continue.
```

Gambar 5. 7 *Tampilan program ketika Sintaks 5.7 Dijalankan*

Dari contoh sintaks 5.7 dan gambar 5.7, dapat dilihat bahwa nilai `i` dan `j` ketika pertama kali ditampilkan merupakan nilai awal yang sesuai dengan nilai ketika di definisikan (`i` = 10, `j` = 20). Saat pemanggilan *Function* `tukar()`, ditampilkan kembali nilai `i` dan `j` ketika sudah ditukar, sehingga nilai `i` menjadi 20 dan nilai `j` menjadi 20. Setelah itu, *Function* `main()` kembali menampilkan nilai `i` dan `j`. Hasilnya adalah nilai `i` 20 dan `j` 10. Mengapa? Karena variabel `i` dan `j` merupakan parameter referensi (ditandai dengan adanya simbol `&` di depan nama parameter), maka apabila nilai di parameter formal berubah, maka di parameter aktual pun ikut berubah.

5.3 Tugas Praktikum

Tugas praktikum sengaja dikosongkan.

PRAKTIKUM 6
REKURSIF

6.1 Tujuan

1. Mahasiswa mampu membuat *function* dengan menggunakan parameter *input* dan parameter *output*.
2. Mahasiswa mampu memahami dan membuat fungsi rekursif.

6.2 Landasan Teori

6.2.1 Fungsi Rekursif

Rekursif adalah salah satu teknik dalam pemrograman, dimana rekursif merupakan sebuah fungsi yang memanggil dirinya sendiri, baik secara langsung maupun tidak langsung (nbloglife, 2013). Secara langsung berarti dalam sebuah fungsi rekursif tersebut terdapat perintah untuk memanggil dirinya sendiri. Secara tidak langsung berarti dalam fungsi rekursif tersebut terdapat perintah untuk memanggil fungsi lain terlebih dahulu sebelum memanggil dirinya sendiri.

Fungsi Rekursif Langsung:

```
Tipe_fungsi nama_fungsi([parameter])
{
   // statement coba([parameter])
   // statement
}
```

Sintaks 6. 1 *Format Penulisan Fungsi Rekursif Langsung*

Fungsi Rekursif Tak Langsung:

```
[ret-val] satu  ([param])
{
   // statement dua([param])
   // statement
}
```

```
[ret-val] dua([param])
{
   // statement dua([param])
   // statement
}
```

Sintaks 6. 2 *Format Penulisan Fungsi Rekursif Tak Langsung*

Secara fungsi, rekursif sama dengan perulangan. Hanya saja rekursif dilakukan melalui fungsi yang memanggil dirinya sendiri. Oleh karena rekursif merupakan fungsi yang memanggil dirinya sendiri, terdapat hal-hal yang perlu diperhatikan dalam penggunaannya. Yaitu pemrogram tidak boleh lupa untuk memberi seleksi kondisi dan perubahan nilai untuk variabel pengendali yang ditujukan sebagai batas perulangan, sama seperti pada perulangan.

Kelemahan dari rekursif adalah diperlukannya tempat penampungan *stack* yang cukup besar. Mengingat setiap proses pemanggilan fungsi, ada beberapa *register* dan beberapa parameter fungsi yang perlu disimpan. Selain itu, karena setiap pemanggilan fungsi *register* dan *memory* harus di *push* ke *stack*, maka setelah pemanggilan fungsi selesai, *register* dan *memory* perlu dikembalikan ke keadaan semula. Hal ini sering disebut sebagai *overhead*.

Walaupun rekursif terlihat lebih buruk dibanding perulangan, faktanya rekursif tak selalu lebih buruk. Adakalanya penggunaan rekursif justru lebih memudahkan penyelesaian suatu masalah dibanding penggunaan perulangan.

6.2.2 Proses Rekursif

Berikut merupakan contoh *Function* rekursif:

```
void rekursif(int x,int y)
{
     if (y == 0) return;
     x++;
     y--;
     printf("Masuk -> x = %d || y = %d \n",x,y);
     rekursi(x,y);
     printf("Keluar -> x = %d || y = %d \n",x,y);
}
```

Sintaks 6. 3 *Contoh Penulisan Fungsi Rekursif*

Sebagai contoh ketika fungsi `rekursif` diatas dipanggil dari fungsi `main()` dengan nilai x = 3 dan y = 3, maka program didalam fungsi rekursif akan mengecek nilai y. apakah y==0? Jika nilai y adalah 0, maka program akan keluar dari fungsi `rekursif` dan kembali ke fungsi `main()`. Namun jika nilai y tidak sama dengan 0, maka nilai x ditambah satu, dan nilai y dikurangi satu. Karena nilai y adalah 3, maka nilai x berubah menjadi 4 dan nilai y berubah menjadi 2. Selanjutnya adalah

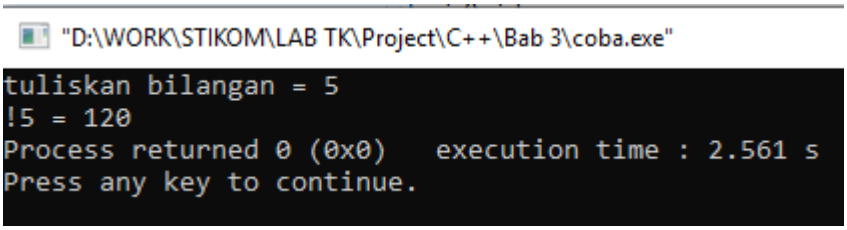
menampilkan nilai x dan y yang telah berubah. Kemudian fungsi `rekursif` memanggil dirinya sendiri dengan nilai x menjadi 4 dan nilai y menjadi 2. Sehingga program kembali ke perintah awal dalam fungsi `rekursif`. Langkah tersebut berulang hingga nilai y menjadi 0 dan tidak memenuhi seleksi kondisi `if (y == 0)`. Kemudian program akan keluar dari fungsi `rekursif` dan mengeksekusi perintah `printf("Keluar -> x = %d || y = %d \n",x,y);` sebelum akhirnya kembali ke fungsi `main()`.

Contoh Menghitung Nilai Faktorial Dengan Rekursif:

```
#include <iostream>

using namespace std;

long factorial (long a) {
    if (a > 1)
        return (a* factorial (a-1));
    else
        return (1);
}

int main() {
    long l;

    cout << "tuliskan bilangan = ";
    cin >> l;
    cout << "!" << l << " = " << factorial(l);

    return 0;
}
```

Sintaks 6. 4 *Contoh Program dengan Fungsi Rekursif*

Tampilan program ketika dijalankan:



Gambar 6. 1 *Tampilan Program Ketika Sintaks 6.4 Dijalankan*

6.3 Tugas Praktikum

Tugas praktikum sengaja dikosongkan.

PRAKTIKUM 7
PEMANTAPAN & PERSIAPAN UTP

7.1 Tujuan

1. Mahasiswa dapat memahami seluruh materi dari praktikum 1 sampai praktikum 6.
2. Mahasiswa dapat sepenuhnya memahami dan menyusun algoritma maupun program menggunakan C++ sesuai dengan materi dari praktikum 1 sampai praktikum 6.

7.2 Landasan Teori

Landasan teori mulai dari praktikum 1 sampai dengan praktikum 6

7.3 Tugas Praktikum

Tugas praktikum sengaja dikosongkan.

PRAKTIKUM 8
ARRAY 1 DIMENSI

8.1 Tujuan

1. Mahasiswa dapat memahami *Array* 1 dimensi.
2. Mahasiswa mampu menyusun program menggunakan struktur *Array* 1 dimensi.

8.2 Landasan Teori

8.2.1 Pengertian *Array*

Ilustrasi

a. Untuk menyimpan sebuah nilai yang mempunyai tipe data tertentu biasanya kita menggunakan sebuah variabel.

   Misalnya:

```
int f1, f2, f3, f4, f5;
```

Sintaks 8. 1 *Sintaks Pendeklarasian Variabel dengan Tipe Data Sama*

   Variabel yang di deklarasi di atas digunakan untuk menyimpan lima data integer dengan nama variabel f1, f2, f3, f4, f5.

b. Jika kita memiliki banyak data yang jumlahnya tidak diketahui atau dapat bertambah kapanpun, penggunaan variabel seperti di atas tidak memungkinkan karena penggunaan variabel di atas hanya menyediakan satu variabel untuk satu data.

Maka *array* dapat menjadi solusi untuk mengatasi masalah di atas.

Array atau larik adalah sekumpulan dari data-data yang memiliki urutan tertentu dengan tipe data yang sama yang menggunakan nama yang sama (sinauarduino (a), 2016). Di dalam sebuah *Array*, posisi dari tiap elemennya ditunjukkan oleh suatu indeks. Berdasarkan dimensinya, terdapat *Array* 1 dimensi, *Array* 2 dimensi, dan *Array* multi-dimensi.

8.2.2 Struktur Dasar *Array* 1 Dimensi

Array 1 dimensi merupakan sebuah deklarasi variabel yang digunakan untuk pendeklarasian variabel dengan tipe data sama dan elemen yang akan diakses hanya melalui 1 indeks. Secara default indeks *array* dimulai dari 0.

Contoh:

Misal seorang siswa mempunyai beberapa nilai ujian yang ingin disusun ke dalam suatu data, ujian pertama mendapat nilai 78, kemudian mendapat nilai 84,97,60. Dalam *Array* dapat disusun sebagai berikut:

ujian[0] = 78;

ujian[1] = 84;

ujian[2] = 97;

ujian[3] = 60;

Di atas adalah proses dalam pemberian nilai pada a*rray* ujian. Akan tetapi, sebelum memberikan suatu nilai pada *array* terlebih dahulu harus mendeklarasikan *array* tersebut.

```
int ujian[4];
```

Sintaks 8. 2 *Sintaks Deklarasi Variabel Array*

Nilai 4 yang ada di dalam tanda kurung menunjukkan banyaknya elemen *array* yang akan disediakan. Dikarenakan indeks yang menunjukkan elemen a*rray* dimulai dari 0, maka elemen *array* ujian yang digunakan dimulai dari 0 hingga 3.

Bentuk umum penulisan:

```
type_data variabel[jumlah_elemen];
```

Sintaks 8. 3 *Bentuk Umum Pendeklarasian Array*

Contoh penulisan program untuk deklarasi *Array*:

```
int nil[5];
```

Sintaks 8. 4 *Bentuk Umum Pendeklarasian Array*

Inisialisasi nilai suatu variabel juga dapat dilakukan secara langsung pada saat deklarasi, misalnya:

```
int nil[5] = { 7,14,21,28,35 };
```

Sintaks 8. 5 *Sintaks Inisialisasi Nilai Awal Array*

Maka penyimpanan ke dalam *array* dapat digambarkan sebagai berikut:

index	0	1	2	3	4
nil	7	14	21	28	35

Gambar 8. 1 *Penyimpanan Array 1 Dimensi*

8.2.3 Menghitung Jumlah Elemen *Array*

Untuk menghitung banyaknya elemen yang terdapat dalam suatu *array* dapat menggunakan fungsi `Sizeof()`. Fungsi ini akan menghitung jumlah *byte* dari *array* yang ada sesuai dengan argumennya, misalnya:

```
int array[] = {166,7,26,82};
cout<<sizeof(array)/sizeof(int);
```

Sintaks 8. 6 *Contoh Penggunaan Sintaks Untuk Menghitung Jumlah Elemen Array*

Fungsi yang dijabarkan pada sintaks 8.6 akan mengembalikan nilai 4, nilai yang sama dengan jumlah elemen *array* yang dimiliki.

8.2.4 Melewatkan *Array* Sebagai Argumen Fungsi

Array dapat dikirim dan dikembalikan oleh fungsi sebagai parameter. Nilai *array* yang dikirim ke dalam fungsi sebagai parameter input dapat dimanipulasi nilai aktualnya.

Contoh:

```
#include <iostream>

using namespace std;

void ubah (int x[]) {
      x[1] = 100;
}

int main() {
      int ujian[] = {90,95,85,75};
      ubah(ujian);
      cout << "Elemen kedua dari Array ujian adalah = " <<
ujian[1] <<endl;
      }
```

Sintaks 8. 7 *Contoh Program Melewatkan Array sebagai Argumen Function*

Tampilan program ketika dijalankan:

```
Elemen kedua dari Array ujian adalah = 100

Process returned 0 (0x0)   execution time : 0.298 s
Press any key to continue.
```

Gambar 8. 2 *Tampilan program ketika Sintaks 8.7 dijalankan*

8.3 Tugas Praktikum

Tugas praktikum sengaja dikosongkan.